Practicum: Temporary File Sharing with Amazon S3 Presigned URLs

Practicum module · Amazon S3 · Node.js / Express · Estimated time: 3 hours

1. General Description

In this practicum you will build **TempShare**, a small web application for sharing files temporarily.

What the application does

- A user selects a file (up to 10 MB) in the browser and uploads it.
- The browser uploads the file **directly to Amazon S3** using a *presigned PUT URL*. The file never passes through your backend server.
- After the upload, the app shows a **shareable link** and a **secret code**.
- Anyone who has both the link and the code can download the file. The backend checks the code, then hands out a *presigned GET URL* that is valid for only 60 seconds.
- **Every upload is deleted within one day.** This is enforced twice: by an S3 lifecycle rule (storage side) and by the backend (link side).

Why this design?

Sending large files through your own server wastes bandwidth and CPU. Presigned URLs let the backend stay small and stateless about file bytes while S3 does the heavy lifting. The bucket stays fully private, and access is granted per file, for a short time, by a server that you control.

1.1 Learning objectives

After completing this practicum you can:

1. Configure a private S3 bucket with CORS, a lifecycle rule and a least-privilege IAM policy.
2. Generate presigned PUT and GET URLs with the AWS SDK for JavaScript v3.
3. Implement a secret-code check using salted hashing and constant-time comparison.
4. Explain why S3 lifecycle expiry alone cannot guarantee a strict 24-hour deadline, and design a second layer to compensate.
5. Identify common security risks of file-sharing services (brute force, link guessing, unrestricted upload).

1.2 Prerequisites

- An AWS account where you may create a bucket and an IAM user.
- Node.js 18 or newer and npm.
- AWS CLI (optional, used for verification).
- Basic knowledge of HTTP, JSON and JavaScript (async/await).

1.3 System architecture

Upload flow

1. Browser sends the file name, type and size to `POST /api/uploads`.

2. Backend creates a random share ID and a random secret code, stores a record (with a hashed code and an expiry time), and returns a presigned PUT URL plus the share link and the code.
3. Browser sends the file with `PUT` straight to S3.
4. Browser shows the share link and the secret code to the uploader.

Download flow

1. Recipient opens `/s/<id>` and enters the secret code.
2. Browser sends the code to `POST /api/shares/<id>/access`.
3. Backend checks that the link exists, has not expired, is not locked, and that the code is correct.
4. Backend returns a presigned GET URL valid for 60 seconds. Browser follows it and downloads the file.

1.4 The two layers of 1-day expiry

Layer	Mechanism	Strength	Weakness
Storage	S3 lifecycle rule: expire objects 1 day after creation	Actually deletes the data, even if your backend is down	Runs on day boundaries (midnight UTC), so an object can live up to about 48 hours, and deletion is asynchronous
Link	Backend stores `expiresAt = now + 24h` and refuses access afterwards	Exact 24-hour cut-off	Only blocks access; it does not delete the object

Together they give a strict 24-hour access window and guaranteed cleanup shortly after. **Discussion question:** what happens if the backend restarts and loses its in-memory records? (See Exercise 1.)

---

2. Part 1: AWS Setup (about 30 minutes)

Use your own bucket name and region. This module uses `tempshare-demo-12345` and `ap-southeast-1` as examples.

2.1 Create the bucket

1. S3 → **Create bucket**.
2. Name: globally unique, e.g. `tempshare-demo-12345`. Region: `ap-southeast-1`.
3. Keep **Block all public access = ON**. Presigned URLs do not need a public bucket.
4. Keep default encryption (SSE-S3).

2.2 Configure CORS

The browser uploads to S3 from your page's origin, so S3 must allow it. Bucket → Permissions → Cross-origin resource sharing (CORS):

```
[
  {
    "AllowedOrigins": ["http://localhost:3000"],
    "AllowedMethods": ["PUT"],
    "AllowedHeaders": ["*"],
    "MaxAgeSeconds": 3000
  }
]
```
In production, replace the origin with your real domain.

This rule is provided in the project as `aws/cors.json`. The AWS CLI expects the same rule wrapped in a `CORSRules` object, provided as `aws/cors-cli.json`:

```
aws s3api put-bucket-cors \
  --bucket tempshare-demo-12345 \
  --cors-configuration file://aws/cors-cli.json
```

2.3 Add the 1-day lifecycle rule

Console: Bucket → Management → Create lifecycle rule. Scope it to the prefix `uploads/`, and enable *Expire current versions of objects* after **1 day**, plus *Delete expired object delete markers or incomplete multipart uploads* after 1 day.

Or with the CLI, use this rule (provided as `aws/lifecycle.json`):

```
{
  "Rules": [
    {
      "ID": "expire-uploads-after-1-day",
      "Status": "Enabled",
      "Filter": { "Prefix": "uploads/" },
      "Expiration": { "Days": 1 },
      "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 1 }
    }
  ]
}
```

```
aws s3api put-bucket-lifecycle-configuration \
  --bucket tempshare-demo-12345 \
  --lifecycle-configuration file://aws/lifecycle.json
```

2.4 Generate IAM User

Open IAM → Users → Create user (e.g. `tempshare-backend`, no console access) → attach the policy → Security credentials → **Create access key** → *Application running outside AWS*. Copy the access key ID and secret; the secret is shown only once.

```
AWS_ACCESS_KEY_ID=<your key> AWS_SECRET_ACCESS_KEY=<your secret> aws sts get-caller-identity
```
The output must show the ARN of `tempshare-backend`.

2.5 Create the IAM policy

IAM → Policies → Create policy → JSON (provided as `aws/iam-policy.json`; replace the bucket name with yours):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::tempshare-demo-12345/uploads/*",
      "Principal": {
        "AWS": "arn:aws:iam::154631643588:user/tempshare-demo-backend"
      }
    }
  ]
}
```

`s3:GetObject` is needed both for download URLs and for the `HeadObject` check the backend performs.

3. Part 2: Project Setup (about 15 minutes)

3.1 Folder structure

```
tempshare/
├── .env
├── .gitignore              (contains: .env and node_modules)
├── README.md               (quick start)
├── package.json
├── server.js
├── public/
│   ├── index.html          (upload page)
│   └── share.html          (download page)
└── aws/
    ├── cors.json           (2.2, console format)
    ├── lifecycle.json      (2.3)
    └── iam-policy.json     (2.4)
```

The `aws/` folder holds the configuration from Part 1 as ready-made files; the application itself needs only the other files. The complete project is available as the starter archive `tempshare.zip`.

3.2 package.json

```
{
  "name": "tempshare",
  "version": "1.0.0",
  "type": "module",
  "scripts": { "start": "node server.js" },
  "dependencies": {
    "@aws-sdk/client-s3": "^3.600.0",
    "@aws-sdk/s3-request-presigner": "^3.600.0",
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
```

```
  }
}
```

Run `npm install`.

3.3 .env (dummy values, replace with yours)

```
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=…
AWS_REGION=ap-southeast-1
S3_BUCKET=tempshare-demo-12345
BASE_URL=http://localhost:3000
PORT=3000
```

The two AWS values above are the dummy examples from AWS documentation. They belong to no account and AWS will reject them (`InvalidAccessKeyId`) until you replace them with the key pair from step 2.4.

---

4. Part 3: Backend (about 40 minutes)

Create `server.js`:

```
import 'dotenv/config';
import express from 'express';
import crypto from 'node:crypto';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import {
  S3Client,
  PutObjectCommand,
  GetObjectCommand,
  HeadObjectCommand,
} from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// Credentials come from AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY automatically.
// WHEN_REQUIRED stops newer SDK versions from adding checksum parameters to
// presigned URLs, which would break browser uploads.
const s3 = new S3Client({
  region: process.env.AWS_REGION,
  requestChecksumCalculation: 'WHEN_REQUIRED',
});

const BUCKET = process.env.S3_BUCKET;
const PORT = process.env.PORT || 3000;
const BASE_URL = process.env.BASE_URL || `http://localhost:${PORT}`;
const TTL_MS = 24 * 60 * 60 * 1000; // links valid for 24 hours
const MAX_SIZE = 10 * 1024 * 1024; // 10 MB
const MAX_ATTEMPTS = 5; // wrong codes before the link is locked
```

```
// id -> { key, filename, size, salt, codeHash, expiresAt, failedAttempts }
// In-memory on purpose (simple). See Exercise 1 for a durable version.
const shares = new Map();

// ---------- helpers ----------
const ALPHABET = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789'; // no 0/O/1/I confusion

function generateCode(length = 8) {
  return Array.from({ length }, () =>
ALPHABET[crypto.randomInt(ALPHABET.length)]).join('');
}

function hashCode(code, salt) {
  return crypto.scryptSync(code, salt, 32);
}

function codeMatches(code, record) {
  const candidate = hashCode(code, record.salt);
  return crypto.timingSafeEqual(candidate, record.codeHash); // constant-time compare
}

// Remove expired records every 10 minutes
setInterval(() => {
  const now = Date.now();
  for (const [id, r] of shares) if (now > r.expiresAt) shares.delete(id);
}, 10 * 60 * 1000);

// ---------- 1. create upload ----------
app.post('/api/uploads', async (req, res) => {
  try {
    const { filename, contentType, size } = req.body;

    if (typeof filename !== 'string' || !filename.trim()) {
      return res.status(400).json({ error: 'filename is required' });
    }
    if (!Number.isInteger(size) || size <= 0 || size > MAX_SIZE) {
      return res.status(400).json({ error: 'size must be between 1 byte and 10 MB'
});
    }

    const type = contentType || 'application/octet-stream';
    const id = crypto.randomBytes(9).toString('base64url'); // unguessable share ID
    const safeName = path.basename(filename).replace(/[^a-zA-Z0-9._-]/g, '_');
    const key = `uploads/${id}/${safeName}`; // must stay under the lifecycle prefix

    const code = generateCode();
    const salt = crypto.randomBytes(16);
    const expiresAt = Date.now() + TTL_MS;

    // ContentType and ContentLength are signed: S3 rejects an upload that differs.
    const uploadUrl = await getSignedUrl(
      s3,
      new PutObjectCommand({
        Bucket: BUCKET,
```

```
        Key: key,
        ContentType: type,
        ContentLength: size,
      }),
      { expiresIn: 300 },
    );

    shares.set(id, {
      key,
      filename: safeName,
      size,
      salt,
      codeHash: hashCode(code, salt), // the plain code is never stored
      expiresAt,
      failedAttempts: 0,
    });

    res.json({
      id,
      code, // shown to the uploader exactly once
      shareUrl: `${BASE_URL}/s/${id}`,
      uploadUrl,
      expiresAt: new Date(expiresAt).toISOString(),
    });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Could not create upload' });
  }
});

// ---------- 2. share page ----------
app.get('/s/:id', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'share.html'));
});

// ---------- 3. verify code and issue download URL ----------
app.post('/api/shares/:id/access', async (req, res) => {
  try {
    const record = shares.get(req.params.id);
    if (!record) return res.status(404).json({ error: 'Link not found' });

    if (Date.now() > record.expiresAt) {
      shares.delete(req.params.id);
      return res.status(410).json({ error: 'This link has expired' });
    }
    if (record.failedAttempts >= MAX_ATTEMPTS) {
      return res.status(429).json({ error: 'Too many wrong codes. Link is locked.'
});
    }

    const code = String(req.body.code || '').trim().toUpperCase();
    if (!codeMatches(code, record)) {
      record.failedAttempts += 1;
      return res.status(403).json({ error: 'Wrong secret code' });
    }
```

```
    // Make sure the browser really finished the upload
    try {
      await s3.send(new HeadObjectCommand({ Bucket: BUCKET, Key: record.key }));
    } catch {
      return res.status(404).json({ error: 'File is not available' });
    }

    const downloadUrl = await getSignedUrl(
      s3,
      new GetObjectCommand({
        Bucket: BUCKET,
        Key: record.key,
        ResponseContentDisposition: `attachment; filename*=UTF-
8''${encodeURIComponent(record.filename)}`,
      }),
      { expiresIn: 60 },
    );

    res.json({ downloadUrl, filename: record.filename, size: record.size });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Could not create download URL' });
  }
});

app.listen(PORT, () => console.log(`TempShare running at ${BASE_URL}`));
```

Checkpoint questions

1. Why is the share ID generated with `crypto.randomBytes` and not a counter?
2. Why do we hash the code instead of storing it, and why use `timingSafeEqual`?
3. What does `MAX_ATTEMPTS` protect against, and what is its weakness (hint: it is per link, not per client)?

---

5. Part 4: Frontend (about 30 minutes)

5.1 public/index.html (upload page)

```
<!doctype html>
<html lang='en'>
<head>
  <meta charset='utf-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1'>
  <title>TempShare</title>
  <style>
    body { font-family: sans-serif; max-width: 480px; margin: 40px auto; padding: 0
16px; }
    code { background: #eee; padding: 2px 6px; border-radius: 4px; font-size: 1.1em;
}
  </style>
</head>
<body>
```

```
  <h1>TempShare</h1>
  <p>Upload a file (max 10 MB). It is deleted after 24 hours.</p>
  <input type='file' id='file'>
  <button id='send'>Upload</button>
  <p id='status'></p>
  <div id='result' hidden>
    <p>Share link: <a id='link' href='#'></a></p>
    <p>Secret code: <code id='code'></code></p>
    <p><small>Send the link and the code separately. The code is shown only
once.</small></p>
  </div>

  <script>
    const $ = (id) => document.getElementById(id);

    $('send').onclick = async () => {
      const file = $('file').files[0];
      if (!file) return;
      const contentType = file.type || 'application/octet-stream';

      try {
        $('status').textContent = 'Requesting upload URL...';
        const res = await fetch('/api/uploads', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ filename: file.name, contentType, size: file.size
}),
        });
        const data = await res.json();
        if (!res.ok) throw new Error(data.error);

        $('status').textContent = 'Uploading to S3...';
        const put = await fetch(data.uploadUrl, {
          method: 'PUT',
          headers: { 'Content-Type': contentType }, // must equal the signed type
          body: file,
        });
        if (!put.ok) throw new Error('S3 upload failed: ' + put.status);

        $('status').textContent = 'Done. Expires at ' + new
Date(data.expiresAt).toLocaleString();
        $('link').textContent = data.shareUrl;
        $('link').href = data.shareUrl;
        $('code').textContent = data.code;
        $('result').hidden = false;
      } catch (err) {
        $('status').textContent = 'Error: ' + err.message;
      }
    };
  </script>
</body>
</html>
```

5.2 public/share.html (download page)

```
<!doctype html>
<html lang='en'>
<head>
  <meta charset='utf-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1'>
  <title>TempShare: Download</title>
  <style>
    body { font-family: sans-serif; max-width: 480px; margin: 40px auto; padding: 0
16px; }
  </style>
</head>
<body>
  <h1>Download file</h1>
  <p>Enter the secret code you received.</p>
  <input id='code' placeholder='Secret code' autocomplete='off'>
  <button id='go'>Download</button>
  <p id='status'></p>

  <script>
    const id = location.pathname.split('/').pop();
    document.getElementById('go').onclick = async () => {
      const status = document.getElementById('status');
      const code = document.getElementById('code').value;
      const res = await fetch('/api/shares/' + id + '/access', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ code }),
      });
      const data = await res.json();
      if (!res.ok) { status.textContent = data.error; return; }
      status.textContent = 'Downloading ' + data.filename + '...';
      window.location = data.downloadUrl; // browser follows the 60-second URL
    };
  </script>
</body>
</html>
```

---

6. Part 5: Run and Test (about 20 minutes)

Start the server with `npm start` and open `http://localhost:3000`. Work through this checklist and record the result of each step in your report.

#	Test	Expected result
1	Upload a 1 MB file	Link and 8-character code are shown
2	In the S3 console, open `uploads/`	One folder named after the share ID containing your file

#	Test	Expected result
3	Open the share link, enter the correct code	File downloads
4	Enter a wrong code 5 times, then the correct one	HTTP 429, link is locked
5	Upload an 11 MB file	Rejected with a size error before reaching S3
6	Copy the `downloadUrl` from the network tab, wait over 60 seconds, open it	S3 returns `AccessDenied` / expired
7	Try to open the object's S3 URL directly (no signature)	`AccessDenied` (bucket is private)
8	Temporarily set `TTL_MS` to 30 seconds, upload, wait, try to download	HTTP 410, link expired

6.1 Troubleshooting

Symptom	Likely cause
`InvalidAccessKeyId`	`.env` still contains the dummy AWS example keys
`SignatureDoesNotMatch` on upload	The `Content-Type` in the `PUT` differs from the one that was signed
Browser shows a CORS error on upload	Origin missing from the bucket CORS rule, or `PUT` not allowed
`AccessDenied` on upload	IAM resource path does not match the `uploads/` key prefix
Upload succeeds but download says file not available	Missing `s3:GetObject` permission (needed by `HeadObject`)
Presigned URL contains `x-amz-checksum` parameters and fails	SDK too new for browser PUT: keep `requestChecksumCalculation: 'WHEN_REQUIRED'`

---

7. Part 6: Security Discussion

Answer these in your report (a short paragraph each):

1. A secret code of 8 characters from a 32-character alphabet has how many combinations? With 5 attempts per link, how likely is a successful guess?
2. Why are the link and the code meant to be sent through *different* channels?
3. The `/api/uploads` endpoint is public. What abuse is possible, and what controls would you add (authentication, rate limiting, file type allowlist, malware scanning)?
4. The presigned GET URL can be forwarded by a recipient during its 60 seconds. Is that acceptable for this application? When would it not be?
5. Why does the lifecycle rule alone not satisfy a requirement such as *files must be inaccessible after exactly 24 hours*?

---

8. Exercises

Exercise 1 (required): durable records. Replace the in-memory `Map` with an Amazon DynamoDB table. Use a **TTL attribute** on `expiresAt` so that expired records are removed automatically. Explain what changes for a deployment with two backend instances.

Exercise 2 (required): countdown. Add a `GET /api/shares/:id` endpoint that returns the remaining lifetime (without revealing the code or the file name) and display a countdown on the share page.

Exercise 3 (choose one):

- **One-time link:** invalidate the share record after the first successful download.
- **Type allowlist:** accept only PDF, PNG, JPEG and ZIP, validated on the server.
- **Per-client rate limit:** limit failed attempts per IP address in addition to per link.

Exercise 4 (bonus): deploy with an IAM role. Run the backend on EC2 (or ECS/Lambda) using an IAM role and remove the access keys from the project completely.

Note: Don’t forget to record the result from this practicum!

9. Clean-up

To avoid charges and leftover access:

1. Empty and delete the bucket.
2. Delete the IAM access key, then the IAM user and the policy.
3. Delete any DynamoDB table created in Exercise 1.
4. Make sure `.env` was never committed to your repository.